VHDL Digital Logic Circuit Design Report

VHDL Homework 1

Item	Information
Course	Digital Logic Circuit Design
Date	October 25, 2026

This report presents five combinational VHDL designs and compares their expected behavior with the available Logisim verification captures. Full VHDL modules are included for each assignment.

1 Objective

The objective of this assignment is to implement fundamental digital logic functions in VHDL and verify their behavior using Logisim-evolution and Questa-compatible source code.

No.	Design	Primary Function
1	Four Input AND and OR Gates	Boolean operations
2	Eight to One Multiplexer	Data selection
3	Seven Segment Alphabet Display	Character encoding
4	Twenty Five Person Voting Machine	Majority decision
5	Thirteen Bit Population Counter	Bit counting and display control

2 Assignment 1 Four Input AND and OR Gates

2.1 Objective

Design a circuit with four binary inputs and two outputs that perform AND and OR operations.

2.2 Design

The inputs are a, b, c, and d. Output out1 is high only when all four inputs are high. Output out2 is high when at least one input is high.

- out1 = a AND b AND c AND d
- out2 = a OR b OR c OR d

2.3 VHDL Implementation

The complete module uses concurrent signal assignments.

```
library ieee;
use ieee.std_logic_1164.all;

entity logic_test is
    port(
        a, b, c, d : in std_logic;
        out1, out2 : out std_logic
    );
end entity logic_test;

architecture bhv of logic_test is
begin
    out1 <= a and b and c and d;
    out2 <= a or b or c or d;
end architecture bhv;
```

2.4 Expected Results and Verification

Inputs abcd	AND out1	OR out2
0000	0	0
0001	0	1
1010	0	1
1111	1	1

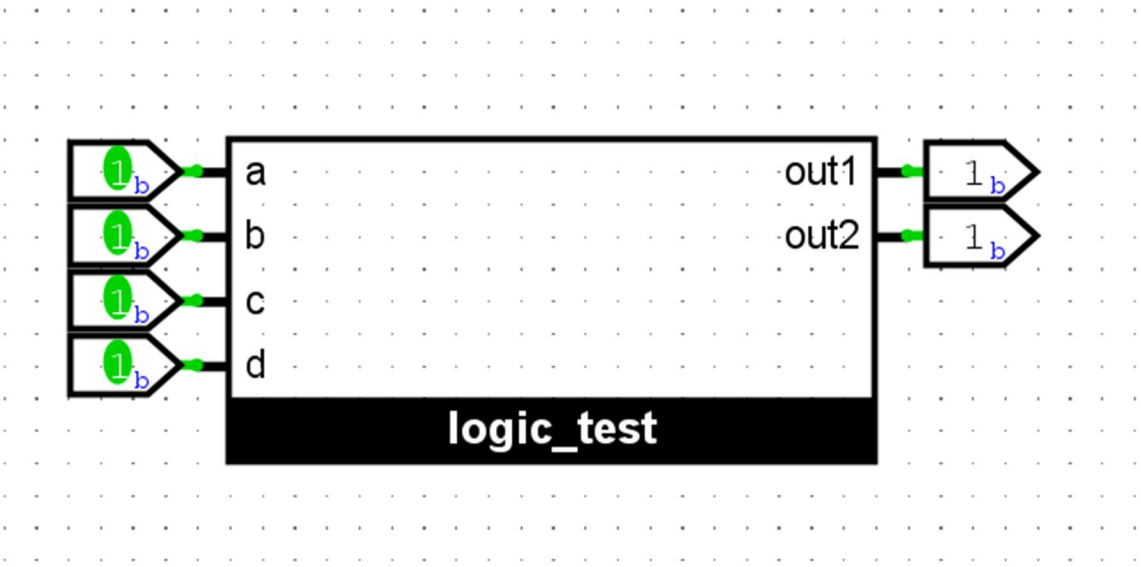


Figure 1 AND and OR verification with a b c d all equal to 1 Outputs out1 and out2 are both 1

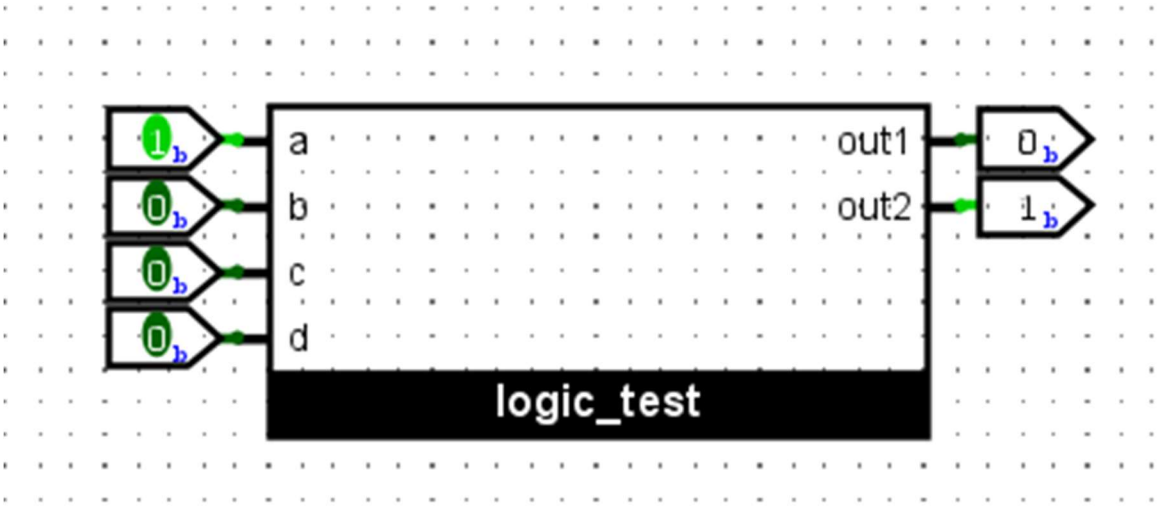


Figure 2 AND and OR verification with a equal to 1 and b c d equal to 0 Outputs are out1 equal to 0 and out2 equal to 1

2.5 Analysis

Both available test cases agree with the Boolean expressions and verify the difference between the AND and OR conditions.

3 Assignment 2 Eight to One Multiplexer

3.1 Objective

Design an eight to one multiplexer that selects one of eight data inputs with a three bit selection signal.

3.2 Design

Inputs a through h provide the data values. Selection input ch determines which data input is transferred to output data.

3.3 VHDL Implementation

The complete module uses a combinational process and a case statement.

```
library ieee;
use ieee.std_logic_1164.all;

entity mux8_1 is
    port(
        a, b, c, d, e, f, g, h : in std_logic;
        ch : in std_logic_vector(2 downto 0);
        data : out std_logic
    );
end entity mux8_1;

architecture bhv of mux8_1 is
begin
    process(a, b, c, d, e, f, g, h, ch)
    begin
        case ch is
            when "000" => data <= a;
            when "001" => data <= b;
            when "010" => data <= c;
            when "011" => data <= d;
            when "100" => data <= e;
            when "101" => data <= f;
            when "110" => data <= g;
            when "111" => data <= h;
            when others => data <= '0';
        end case;
    end process;
end architecture bhv;
```

3.4 Expected Results and Verification

Selection ch	Selected Input
000	a
001	b
010	c
011	d
100	e
101	f
110	g
111	h

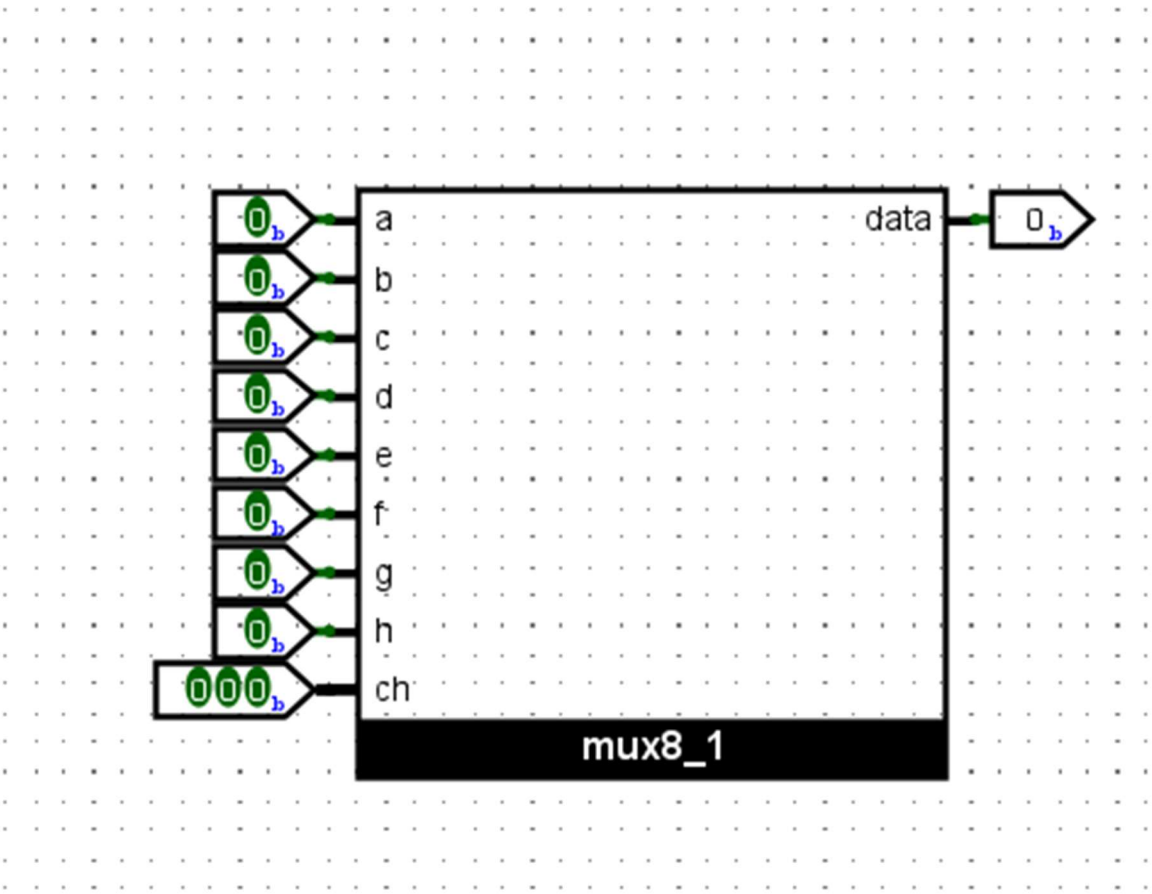


Figure 3 Multiplexer verification for ch equal to 000 with all data inputs low Output data is 0

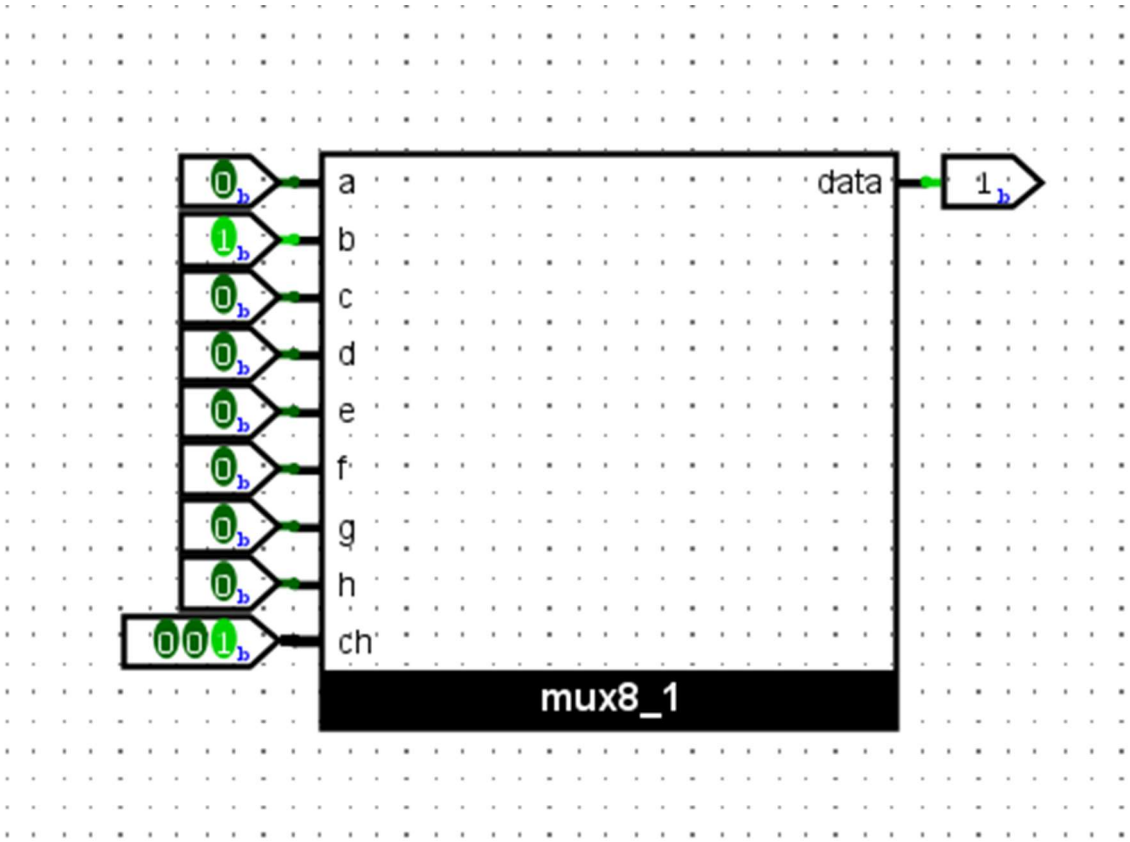


Figure 4 Multiplexer verification for ch equal to 001 with input b high Output data is 1

3.5 Analysis

The captures verify selection values 000 and 001. These results agree with the case statement. Additional captures for selection values 010 through 111 are still required for complete experimental coverage.

4 Assignment 3 Seven Segment Alphabet Display

4.1 Objective

Display the letters a, b, c, d, f, and g on an active high seven segment display.

4.2 Design

The output vector uses the bit order a b c d e f g dp. A value of 1 activates the corresponding segment. Because seven segment displays cannot reproduce every letter exactly, lower case forms are used where they are easier to distinguish.

4.3 VHDL Implementation

The character patterns follow the display convention used in this assignment. The pattern 11111010 is treated as the lowercase letter a.

```
library ieee;
use ieee.std_logic_1164.all;

entity segment_letters is
    port(
        ch : in std_logic_vector(2 downto 0);
        display : out std_logic_vector(7 downto 0)
    );
end entity segment_letters;

architecture bhv of segment_letters is
begin
    process(ch)
    begin
        case ch is
            -- Bit order: a b c d e f g dp
            when "000" => display <= "11111010"; -- a
            when "001" => display <= "00111110"; -- b
            when "010" => display <= "00011010"; -- c
            when "011" => display <= "01111010"; -- d
            when "100" => display <= "10001110"; -- f
            when "101" => display <= "11110110"; -- g
            when others => display <= "00000000";
        end case;
    end process;
end architecture bhv;
```

4.4 Expected Results and Verification

Selection ch	Displayed Letter	Pattern a b c d e f g dp
000	a	11111010
001	b	00111110
010	c	00011010
011	d	01111010
100	f	10001110
101	g	11110110

Selection ch	Displayed Letter	Pattern a b c d e f g dp
110	OFF	00000000
111	OFF	00000000

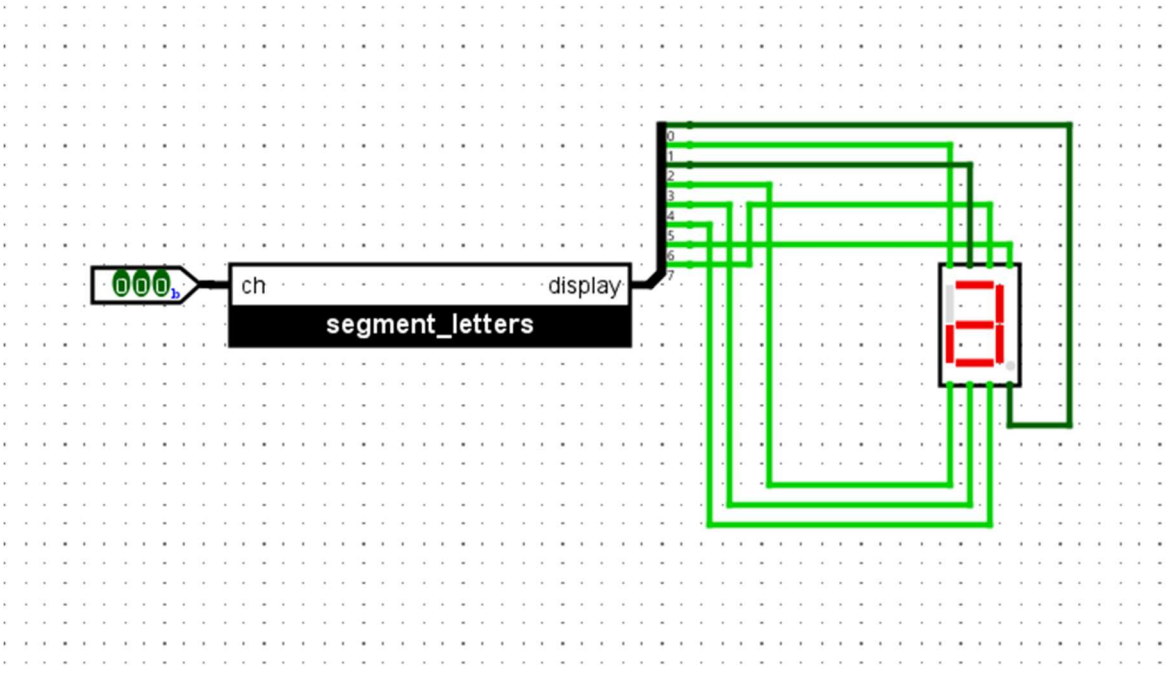


Figure 5 Seven segment verification for ch equal to 000 The displayed shape represents lowercase a under the assignment convention

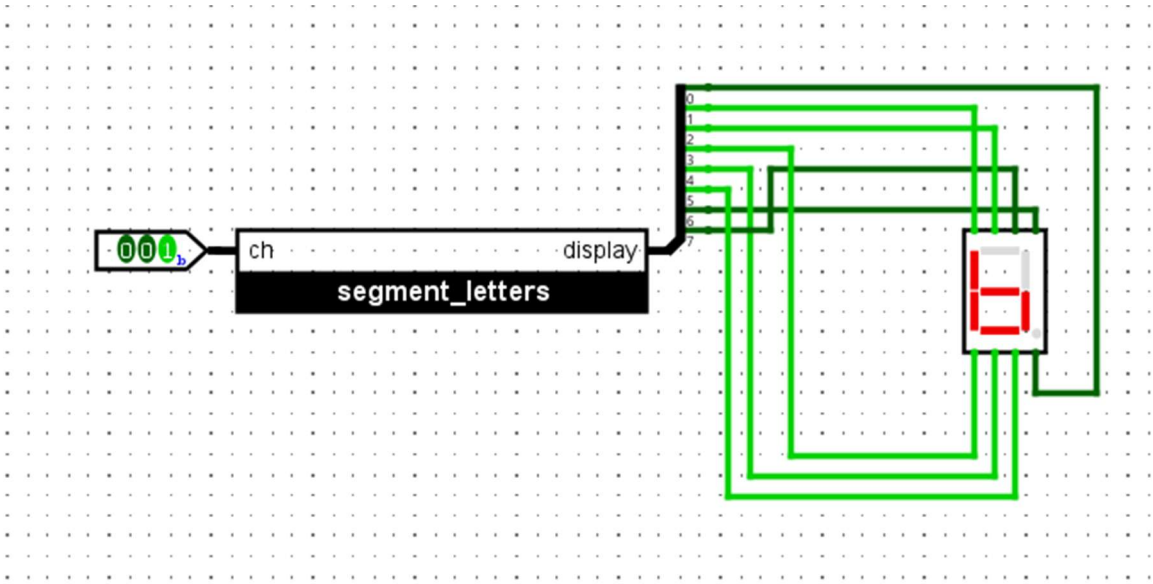


Figure 6 Seven segment verification for ch equal to 001 The displayed shape represents lowercase b under the assignment convention

4.5 Analysis

The available captures verify the assignment conventions for a and b. Additional captures for c, d, f, and g are required for complete experimental coverage.

5 Assignment 4 Twenty Five Person Voting Machine

5.1 Objective

Design a voting circuit that approves a proposal when at least 13 of 25 participants vote in favor.

5.2 Design

Input vector vt contains 25 votes. A high bit represents approval. The result output is high when the approval count is 13 or greater.

5.3 VHDL Implementation

A loop counts the high bits, and a conditional statement applies the majority threshold.

```
library ieee;
use ieee.std_logic_1164.all;

entity vote25 is
    port(
        vt : in std_logic_vector(24 downto 0);
        result : out std_logic
    );
end entity vote25;

architecture bhv of vote25 is
begin
    process(vt)
        variable sum : integer range 0 to 25;
    begin
        sum := 0;

        for i in 0 to 24 loop
            if vt(i) = '1' then
                sum := sum + 1;
            end if;
        end loop;

        if sum >= 13 then
            result <= '1';
        else
            result <= '0';
        end if;
    end process;
end architecture bhv;
```

5.4 Expected Results and Verification

Approval Votes	Rejection Votes	Result
0	25	0
12	13	0
13	12	1
20	5	1
25	0	1

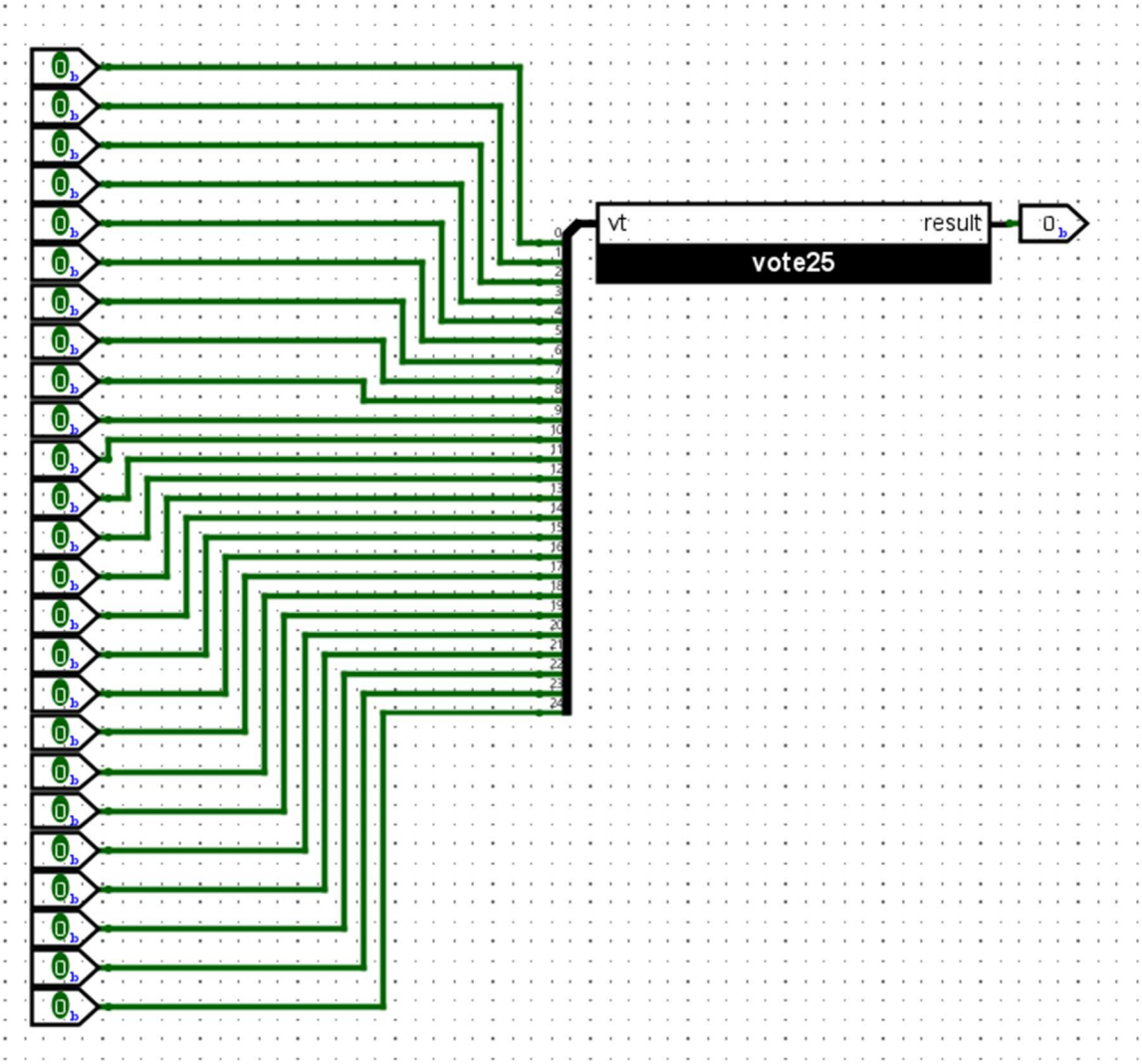


Figure 7 Voting circuit with zero approval votes Result is 0

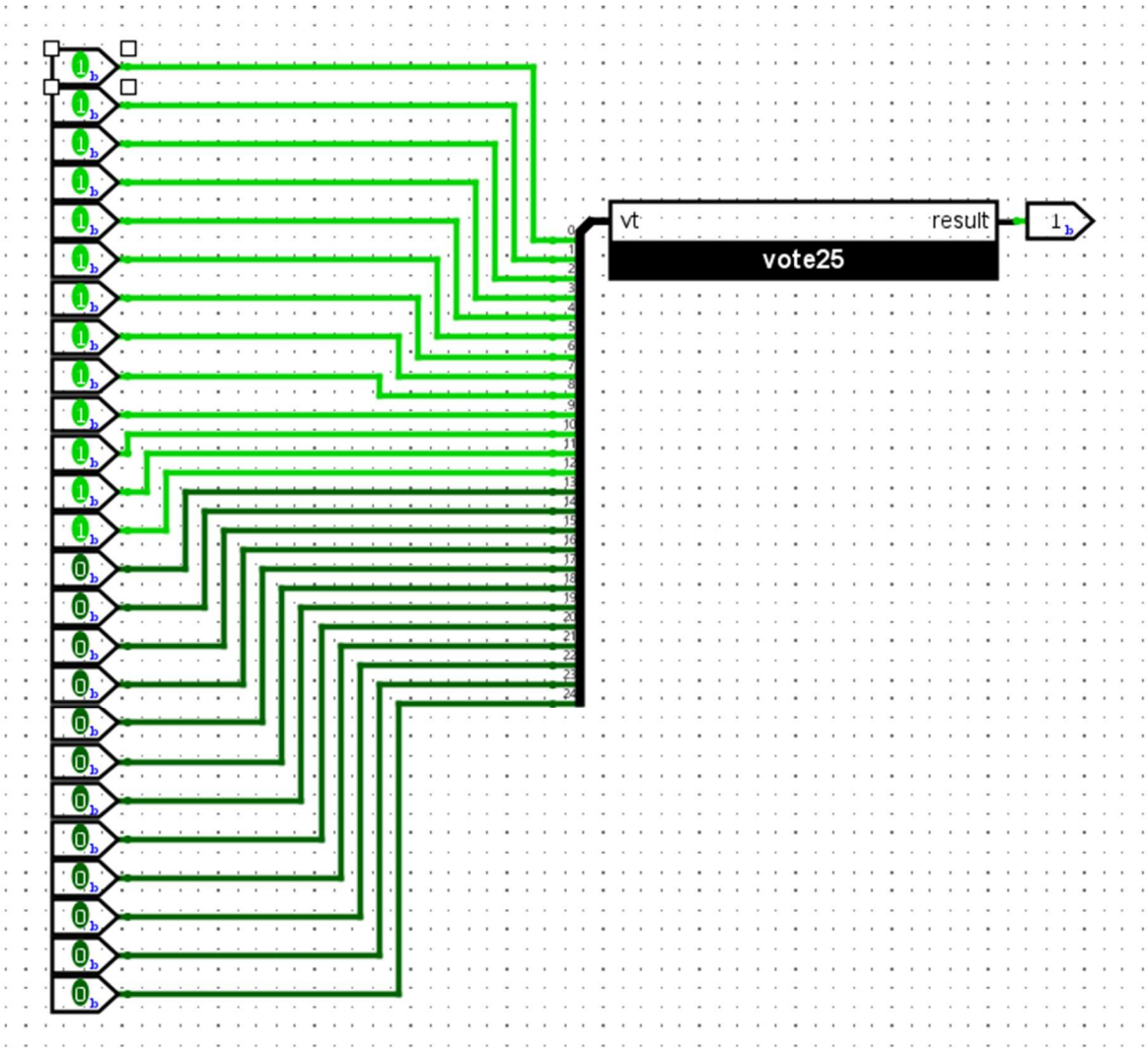


Figure 8 Voting circuit with 13 approval votes Result is 1

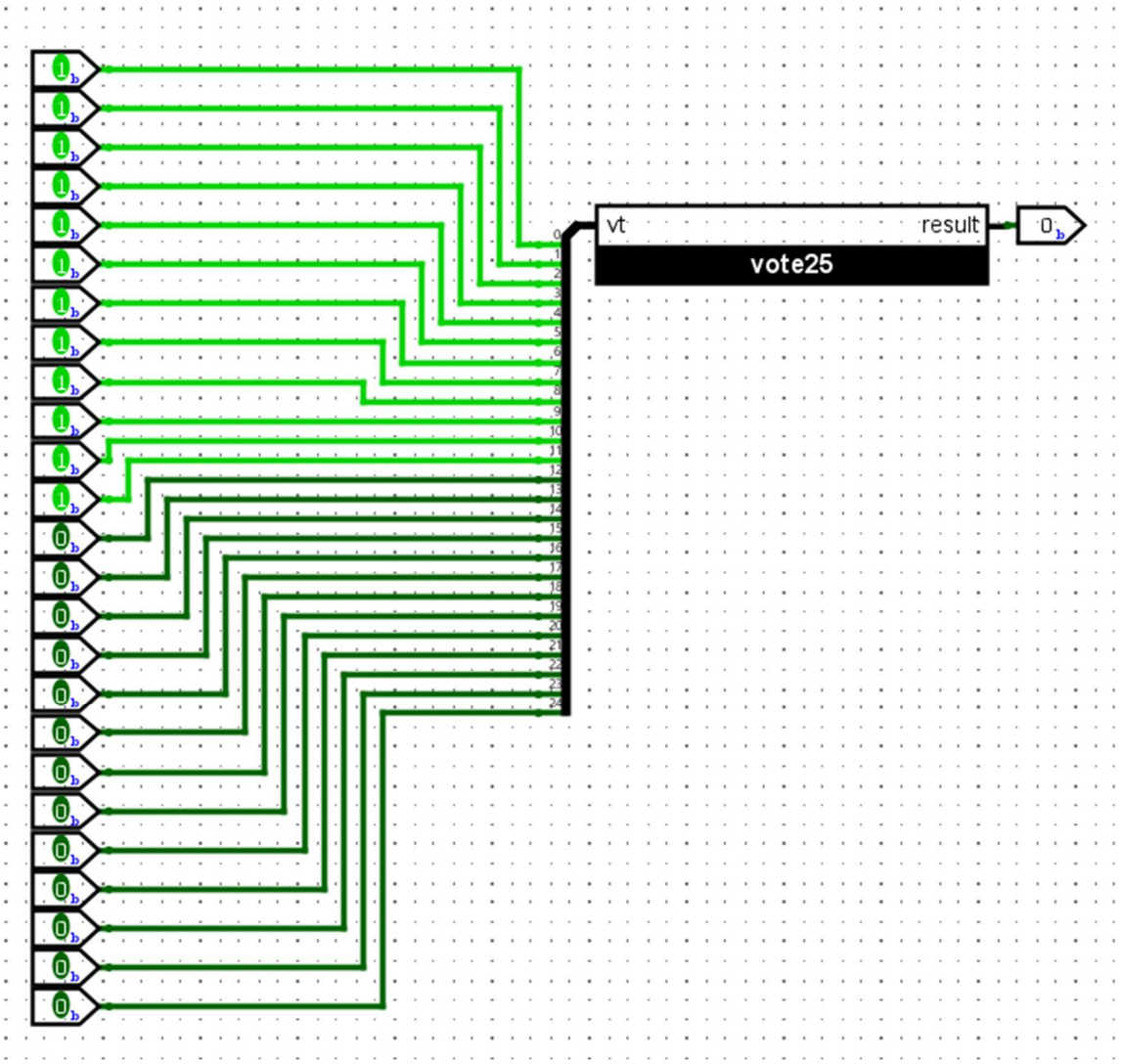


Figure 9 Voting circuit with 12 approval votes Result is 0

5.5 Analysis

The 12 vote and 13 vote captures verify the critical majority boundary. The output changes from 0 to 1 exactly at 13 approval votes.

6 Assignment 5 Thirteen Bit Population Counter

6.1 Objective

Count the number of high bits in a 13 bit input vector and display the count when enable input en is high.

6.2 Design

The circuit counts the high bits in vt. When en is 0, all display segments are off. When en is 1, counts 0 through 9 are displayed as decimal digits and counts 10 through 13 are displayed as A, b, C, and d. Counts 14 and 15 are encoded for completeness but cannot occur with a 13 bit input vector.

6.3 VHDL Implementation

The complete module combines a counting loop, enable control, and a seven segment case statement.

```
library ieee;
use ieee.std_logic_1164.all;

entity counter7seg is
    port(
        vt      : in std_logic_vector(12 downto 0);
        en      : in std_logic;
        display : out std_logic_vector(7 downto 0)
    );
end entity counter7seg;

architecture bhv of counter7seg is
begin
    process(vt, en)
        variable sum : integer range 0 to 15;
    begin
        sum := 0;

        for i in 0 to 12 loop
            if vt(i) = '1' then
                sum := sum + 1;
            end if;
        end loop;

        if en = '0' then
            display <= "00000000";
        else
            case sum is
                -- Bit order: a b c d e f g dp
                when 0  => display <= "11111100";
                when 1  => display <= "01100000";
                when 2  => display <= "11011010";
                when 3  => display <= "11110010";
                when 4  => display <= "01100110";
                when 5  => display <= "10110110";
                when 6  => display <= "10111110";
                when 7  => display <= "11100000";
                when 8  => display <= "11111110";
                when 9  => display <= "11110110";
                when 10 => display <= "11101110"; -- A
                when 11 => display <= "00111110"; -- b
                when 12 => display <= "10011100"; -- C
                when 13 => display <= "01111010"; -- d
                when 14 => display <= "10011110"; -- E
                when 15 => display <= "10001110"; -- F
                when others => display <= "00000000";
            end case;
        end if;
    end process;
end architecture bhv;
```

6.4 Expected Results and Verification

Number of High Bits	Enable en	Display
0	1	0
5	1	5
9	1	9
10	1	A
11	1	b
12	1	C
13	1	d
Any	0	OFF

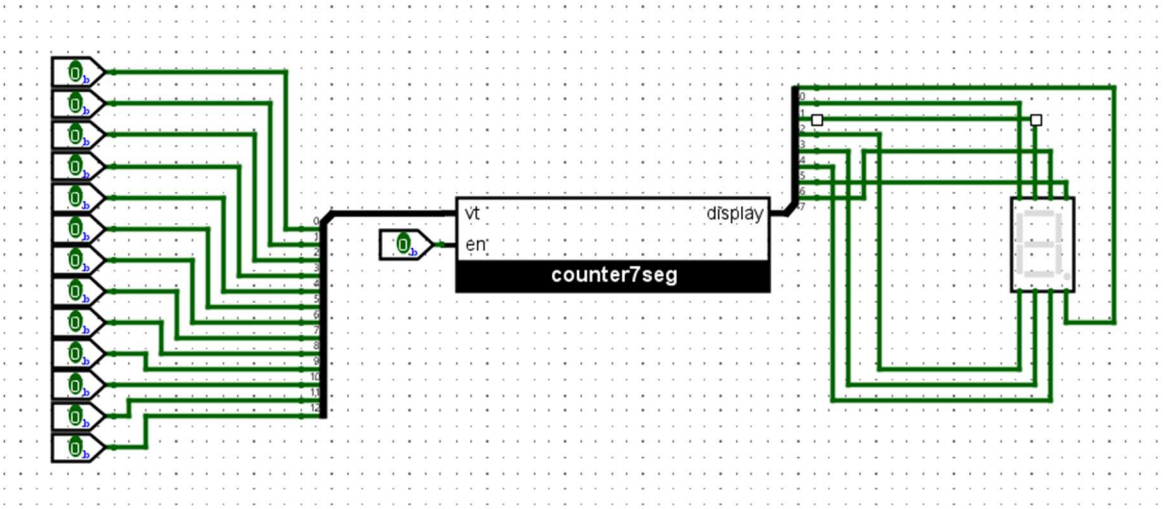


Figure 10 Population counter with en equal to 0 The display is off

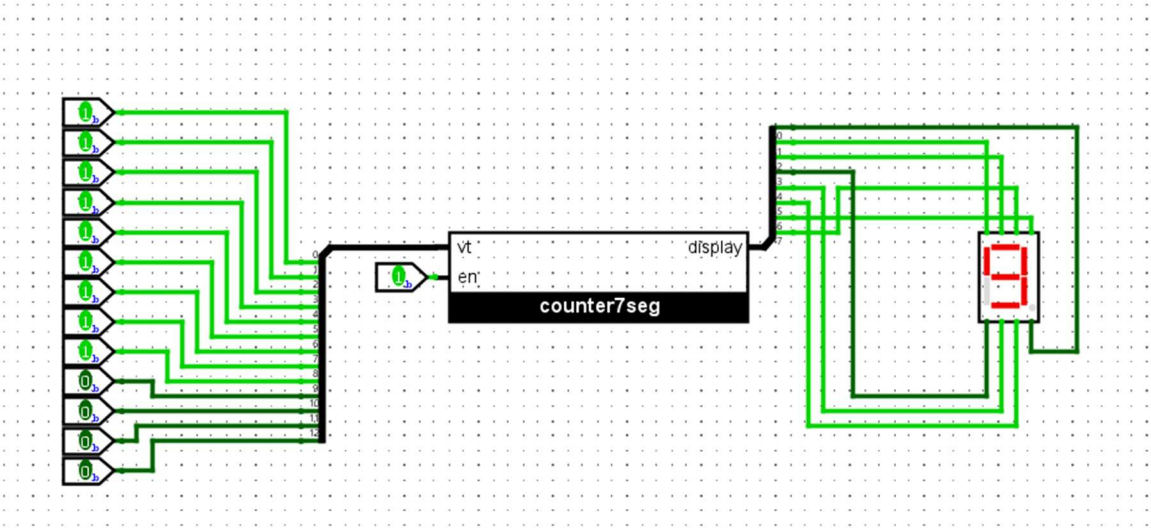


Figure 11 Population counter with en equal to 1 and nine active inputs The display shows 9

6.5 Analysis

The captures verify the disabled condition and the enabled count of 9. Additional captures for counts 10 through 13 are required to verify the hexadecimal character patterns.

7 Conclusion

The five VHDL modules demonstrate concurrent assignments, combinational processes, case statements, loops, and conditional decisions. The available captures verify the AND and OR circuit, two multiplexer selections, the voting threshold, the population counter disable condition, and the count of 9.

Additional captures for the remaining multiplexer selections, the remaining seven segment letters, and population counts 10 through 13 would provide complete experimental coverage before the final PDF is submitted.